Team name: COCO Puffs

Group members: Adam Hetherwick, Millen Bisrat, Luis Olea, Joy Zhang

Problem

Farmers and related animal welfare organizations occasionally face issues with headcounts, herd locations, and multi-species detection. This is due to a lack of reliable information or technology to count or classify their herds. Drones and camera traps can easily capture images of animals; however, this can result in hundreds, if not thousands, of data points, for which hand-labeling such images can be time-consuming and prone to manual input error.

Implementation of a variety of computer vision models for the purpose of object detection and species classification can mitigate this problem and provide accurate findings for refined animal crops, headcounts, and species identification. This methodology of multi-object detection can extend to a multitude of facets and remains a relevant skill set to possess in the field of computer vision. It can be much more accurate than human headcounts and species identification.

Motivation

As UC Davis students, we understand that agriculture is a vital part of any functioning society, and efficient management of such organizations sustains a healthy future. Agriculture and related practices are a major part of the California economy, and nearly all people are involved either as a producer, consumer, or both. Maximizing efficiency in such an important field will ultimately help distribute, produce, and manage products. This, after a few in between steps, can literally help put food on people's tables. This is precisely why people should be interested in this project; because it accelerates a process they inevitably take part in.

The problem we are solving is that farmers and participants in agricultural management need quick and easy information regarding their cattle for example location, quantity, and species distribution. This can be achieved by drones or automated camera traps that capture images of animals either upon movement or by timestamp, and passed into a computer vision model to detect species and refine image crops via multi-object detection. This is a multi- object detection format because each image has roughly 1-5 animals present.

The potential for this project is a fully automated drone based system that locates the herd, and informs the user the location of the herd, number of animals, and refined image crops of each so that the user can inspect for quality assurance. Multi-object detection is a relevant skill for computer vision professionals and methodologies implemented during this project can be applied to other related datasets.

Dataset

We will be using Microsoft COCO 2017 (Common Objects in Context), a publicly released dataset of everyday scenes. Our project will focus on the 10 animal subclasses (bear, bird, cat, cow, dog, elephant, giraffe, horse, sheep, zebra, and potentially humans). Images can contain several animals and other objects, which makes this dataset useful for learning both where an animal is and what category it belongs to.

For this project, we need a bounding box and a class label for each animal. Both are already included in COCO's instance annotations, so we do not need to collect new images or label them by hand. We will then convert the box coordinates and class IDs into the format needed by YOLOv8s.

The full COCO 2017 dataset has 118,287 training images and 5,000 validation images across 80 object classes. Our selected animal subset contains 23,989 unique training images with 61,787 individually annotated animal boxes, plus 1,016 validation images. This gives us more than 20,000 labeled training images. Images containing multiple animal classes are counted only once in these totals. We will keep the official training and validation sets separate and divide the animal validation images into development and final test subsets.

One drawback is that the classes are uneven: dogs appear in 4,385 training images, while bears appear in 960. Animals can also be small, partly hidden, or overlapping, which makes their boxes and categories harder to predict. The class imbalance can be mitigated by undersampling techniques for the more represented classes, or image augmentation on the underrepresented classes.

Method

We tentatively plan to train the YOLOv8s model both from scratch, and from a pretrained model in which we just train the final few layers. This entails a convolutional backbone with a decoupled head that predicts box coordinates and classification. These models have an estimated 11 million parameters, which is small enough to fit within the limits of Colab's T4 GPU and fast enough to train in a reasonable time. In order to prevent data leakage on the pretrained model, we must evaluate only on the *val2017* COCO subset, with weights coming from *train2017*. We estimate training will take 10 to 20 minutes per epoch on T4. YOLOv8s combine a box regression loss for localization and binary cross-entropy for classification. The losses send gradients back to the model, which minimize losses as a weighted sum. The loss weights are precalculated from COCO, with box loss weighted approximately 15 times more than class loss. We can attempt to fine tune the weights ourselves but if model training takes a long time, then evaluation of different models with different loss weights can be timely.

We will evaluate our model with the standard COCO detection metrics using *pycocotools* with descriptions seen in Table 1: mAP@0.5, mAP@[0.5:0.95] and AP (average precision) per class. We will also make a confusion matrix to further examine model behavior. Loading example

predictions, particularly incorrect instances can help us assess issues and generate potential fixes. Due to the class imbalance, we will pay closer attention to per-class AP than mAP to ensure there are no hidden poor results in a smaller class like “bear”. The original COCO paper reported results using a Deformable Parts Model (DPM) that we will compare our results to. DPMs are a bit outdated in the CV world, utilizing a latent support vector machine (SVM) as a classifier. We expect our CNN to outperform the DPM by a wide margin. Since the original model used all 80 COCO categories and its own splits and protocol, we will take the published categorical AP for our ten animal classes and compare against YOLOv8s’ AP.

Metric	**Extended Name**	**Description**	**Notes**
AP	Average precision	Area under precision-recall curve (AUC)	$0 \leq AUC \leq 1$ Closer to 1 the better. One value per class.
mAP	Mean AP	Average AP across classes	$0 \leq mAP \leq 1$ Closer to 1 the better.
mAP@0.5	Mean AP with IoU (intersection over union) ≥ 0.5	Gauging if box is roughly in the right place	$0 \leq mAP@0.5 \leq 1$ Closer to 1 the better.
mAP@[0.5:0.95]	Mean AP with IoU (intersection over union) ≥ 0.5, 0.55, …, 0.95	Mean AP calculated over 10 thresholds. Rewards tighter boxes	COCO’s primary metric

Table 1: Displays our model evaluation metrics and descriptions